DATA TYPES

Objectives:

- Define Table
- Data Types
  - Primary Key, Foreign Key, NULL
  - Column Definition

Tables

- **Tables are database objects that stores data in a collection of rows and columns**

- **A table definition is a collection of columns.**

- **Each row represents a unique record, and each column represents a field within the record.**

Naming Conventions for Tables and Columns

- First character must be a letter or an underscore (_)
- Subsequent characters can be letters, digits, @, $, #, _
- Length up to 128
- Avoid using reserved words

Naming Conventions for Tables and Columns

- **Spaces and other symbol may be used if delimited by a double quotation marks or brackets [ ]. This is not recommended as the delimiter must always be used when referencing that name**
  - **ex. “First Name” or [First Name]**

Data Types

- **is an attribute that specifies the type of data that the column can store**

Column Data Types

Character Data

- **Character data consists of any combination of letters, symbols, and numeric characters.**

Column Data Types

Character Data

char[(n)]

- **Fixed-length non-Unicode character data with length of *n* bytes. *n* must be a value from 1 through 256.**

Column Data Types

Character Data

nchar[(n)]

- **Fixed-length Unicode character data with length of *n* bytes. *n* must be a value from 1 through 65,536. SQL Server allocates double the space internally for this type.**

Column Data Types

Character Data

varchar[(n)]

- Variable-length non-Unicode character data with length from 1 through 8,000. If no column size define - that is, varchar(), the length will default to 1
- can use another setting that can exceed the 8,000-character limit, by defining the data type with the constant *max.*

Column Data Types

Character Data

nvarchar[(n)]

- The nvarchar type is defined in a similar way to varchar, except it uses Unicode and therefore doubles the amount of space required to store the data

Character Data Types

- **CHAR, NCHAR are fixed length**
- **VARCHAR, NVARCHAR are variable length**
- **Character data is delimited with single quotes**

Data Type	Range	Storage
CHAR(n), NCHAR(n)	1-8000 characters	n bytes, padded
VARCHAR(n), NVARCHAR(n)	1-8000 characters	n bytes
VARCHAR(MAX), NVARCHAR(MAX)	1-2^31-1 characters	Actual length + 2

Column Data Types

Date and Time Data

- **Date and time data consists of valid date or time combinations.**

Column Data Types

Date and Time Data

date

- hold a date from January 1, AD 1 through December 31, 9999. The format is YYYY-MM-DD.

Column Data Types

Date and Time Data

datetime

- **Date and time data from January 1, 1753 through December 31, 9999, to an accuracy of 0.00333 second, storage is 4 bytes**

Column Data Types

Date and Time Data

datetime2

- **Date and time data from January 1, AD 1 through December 31, 9999, to an accuracy of 100 nanosecs , storage is 6-8 bytes**

Column Data Types

Date and Time Data

smalldatetime

- **Date and time data from January 1, 1900, through June 6, 2079, with accuracy to the minute, storage is 4 bytes.**

Column Data Types

Date and Time Data

datetimeoffset

- store a time relative to a specific date and time zone. If you want to store 6 p.m. on March 24, 2008 in US Eastern Standard Time, the value would be stored as 2008-03-24 13:00:00 +05:00

Column Data Types

Date and Time Data

time

- hold a time based on the 24-hour clock. Format is hh:mm:ss[.nnnnnnn].

Date and Time Data Types

- Older versions of SQL Server supported only DATETIME and SMALLDATETIME
- DATE, TIME, DATETIME2, DATETIMEOFFSET introduced in SQL Server 2008
- SQL Server 2012 adds new functionality for working with date and time data types

Data Type	Storage (bytes)	Date Range	Accuracy	Recommended Entry Format
DATETIME	8	January 1, 1753 to December 31, 9999	3-1/3 milliseconds	'YYMMDD hh:mm:ss:nnn'
SMALLDATETIME	4	January 1, 1900 to June 6, 2079	1 minute	'YYMMDD hh:mm:ss:nnn'
DATETIME2	6 to 8	January 1, 0001 to December 31, 9999	100 nanoseconds	
DATE	3	January 1, 0001 to December 31, 9999	1 day	'YYYY-MM-DD'
TIME	3 to 5		100 nanoseconds	'hh:mm:ss:nnnnnnn'
DATETIMEOFFSET	8 to 10	January 1, 0001 to December 31, 9999	100 nanoseconds	'YYY-MM-DD hh:mm:ss:nnnnnnn [+|-]hh:mm'

Column Data Types

Numeric Data

- **Numeric data consists of numbers only. Numeric data includes positive and negative numbers, decimal and fractional numbers, and whole numbers (integers).**

Column Data Types

Integer Data

Integer data consists of negative or positive whole numbers, such as -15, 0, 5, and 2509.

Column Data Types

Integer Data

bigint

- Integer (whole number) data from -2^63 (-9223372036854775808) through 2^63-1 (9223372036854775807). Storage size is 8 bytes.

Column Data Types

Integer Data

int

- **Integer (whole number) data from -2^31 (-2,147,483,648) through 2^31 - 1 (2,147,483,647). Storage size is 4 bytes.**

Column Data Types

Integer Data

smallint

- **Integer data from -2^15 (-32,768) through 2^15 - 1 (32,767). Storage size is 2 bytes.**

Column Data Types

Integer Data

tinyint

- Integer data from 0 through 255. Storage size is 1 byte.

Column Data Types

Integer Data

bit

- This can contain only a 1 or a 0 as a value (or null, which is no value). It's useful as a status bit - on/off, yes/no, or true/false, for example

Column Data Types

Decimal Data

- **Consists of data that is stored to the least significant digit**

Column Data Types

Decimal Data

decimal[(p[,s])] and numeric[(p[,s])]

- **Fixed precision and scale numbers. When maximum precision(38) is used, valid values are from - 10^38 +1 through 10^38 - 1**

Column Data Types

Decimal and Numeric

Precision

- The number of digits in a numeric column.

Scale

- The number of digits to the right of a decimal point in a numeric column.

Column Data Types

Decimal Data

float

- hold very

large numbers in the range of –
1.79E+308 through 1.79E+308

real

- can store only numbers in the range of –
3.40E+38 through 3.40E+38

Column Data Types

Monetary Data

Monetary data represents positive or negative amounts of money

Column Data Types

Monetary Data

money

- data type to store values in the range from -922,337,203,685,477.5808 through +922,337,203,685,477.5807

Column Data Types

Monetary Data

smallmoney

- data type to store values in the range from -214,748.3648 through 214,748.3647

Numeric Data Types

Data type	Range	Storage (bytes)
tinyint	0 to 255	1
smallint	-32,768 to 32,768	2
int	2^31 (-2,147,483,648) to 2^31-1 (2,147,483,647)	4
Bigint	-2^63 (-9,223,372,036,854,775,808) to 2^63-1 (9,223,372,036,854,775,807)	8
bit	1, 0 or NULL	1
decimal/numeric	- 10^38 +1 through 10^38 – 1 when maximum precision is used	5-17
money	-922,337,203,685,477.5808 to 922,337,203,685,477.5807	8
smallmoney	- 214,748.3648 to 214,748.3647	4

Numeric Data Types

- Decimal/numeric are functionally equivalent and use precision and scale parameters:

```
DECLARE @mydecimal AS DECIMAL(8,2),
                @myNumeric as NUMERIC (8,2)
```

- Approximate Numeric

Data Type	Range	Storage (bytes)
float(n)	- 1.79E+308 to -2.23E-308, 0 and 2.23E-308 to 1.79E+308	Depends on value of n, 4 or 8 bytes
real	- 3.40E + 38 to -1.18E - 38, 0 and 1.18E - 38 to 3.40E + 38	4

- float(24) is the ISO synonym for float
  - In float(n), n is the number of bytes used to store the mantissa of the float number in scientific notation
- Values of float are truncated when converted to integer types

NULL Value

- **A null value, or NULL, is not the same as zero (0), blank, or a zero-length character string such as “”**

- **NULL means that no entry has been made.**

IDENTITY Column

- use to create a column that contains system-generated sequential values to identify each row inserted into a table

- this value is based on a seed value and an increment value

IDENTITY Column

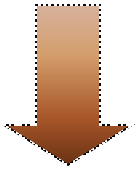
- there can only be one identity column per table
- NULL values are not allowed
- It can be used with the following datatype: *tinyint, smallint, int, bigint or numeric/decimal* with a scale of zero

PRIMARY KEY

- Column or set of columns that uniquely identify each row in a table
- It does not allow null values
- No two rows can have the same primary key value

PRIMARY KEY

Primary Key

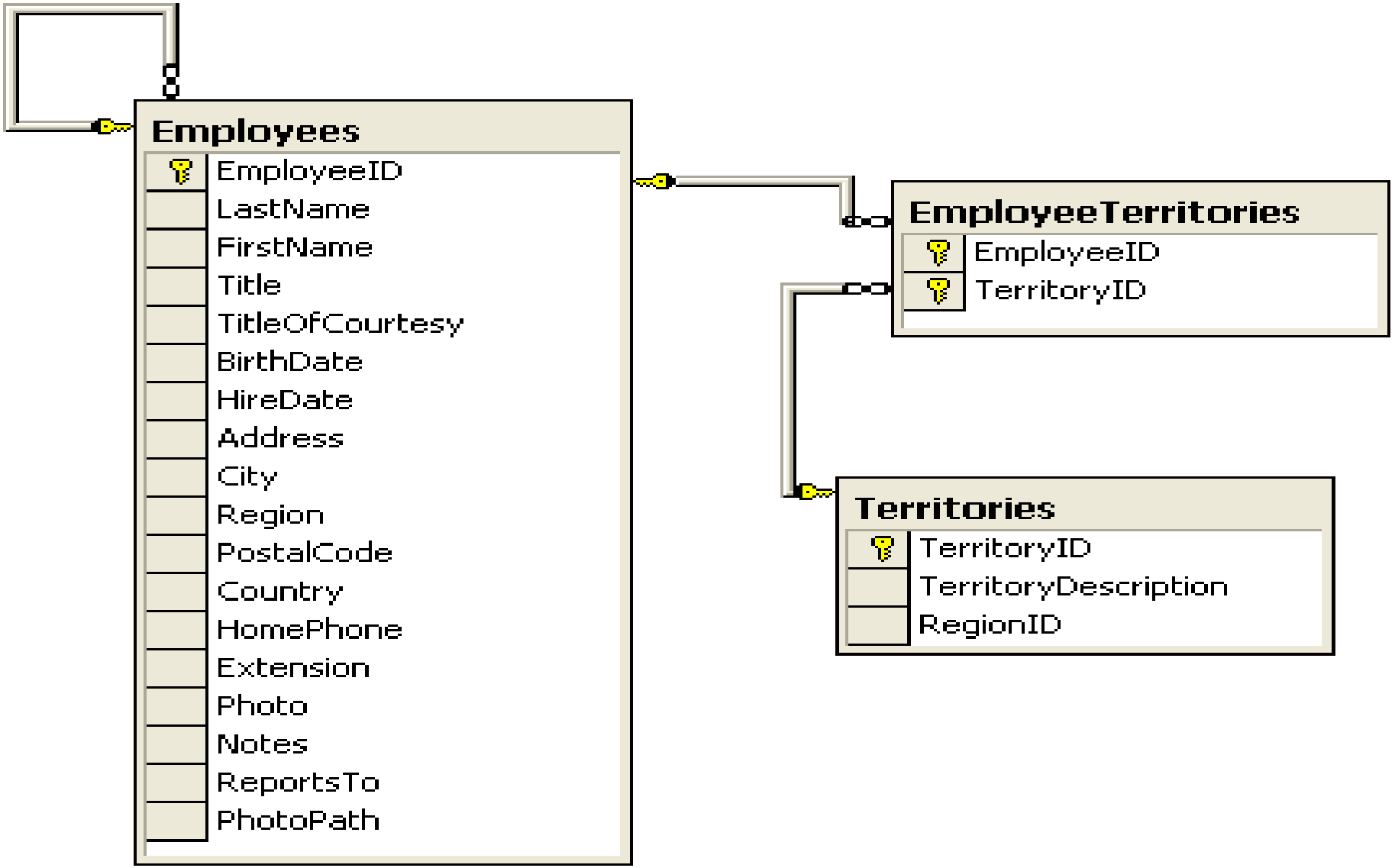


au_id	title_id	au_ord	royalty
172-32	PS333	1	100
213-40	BU1032	2	40
213-46	BU2075	1	100
238-95	PC1035	1	500

FOREIGN KEY

- **Column or set of columns whose values match the primary key (PK) in the same or another table**

FOREIGN KEY


Employees
EmployeeID
LastName
FirstName
Title
TitleOfCourtesy
BirthDate
HireDate
Address
City
Region
PostalCode
Country
HomePhone
Extension
Photo
Notes
ReportsTo
PhotoPath
EmployeeTerritories
EmployeeID
TerritoryID
Territories
TerritoryID
TerritoryDescription
RegionID

Column Definition

Column Name

- Should be meaningful so as to describe the column content.

Data Type

- Any one of 25 possible definitions provides the basis for the data a column will contain.

Column Definition

Length

- For many of the data types, the length is predetermined. Specify a length for character, binary and variable-length data

Allow Nulls

- You can provide an indicator for allowing NULL content for any variable

Column Definition

Primary Key

- Enforces unique content for a column and can be used to relate other tables. Must contain a unique non-NULL value

Description

- Provides an explanation of the column for documentation purposes.

Column Definition

Default Value

- Provides a value for a column when one is not explicitly given during data entry.

Identity

- Inserts a value automatically into a column, based on seed and increment definitions.

Column Definition

Identity Seed

- Provides the starting value for an Identity column.

Identity Increment

- Defines how an Identity will increase or decrease with each new row added to a table